Assignment 3: Testing, UX, and Development Plan

September 2026

1 Project Identification

Team Name	Big O(oof)
Team Members	Gayatri Taneja, Shivanshi Singh
Roll Numbers	1020241849, 1020241319
Assigned Project	Interactive Delhi Metro Route Planner and Visualiser
Repository Link	`https://github.com/Ashoka-CS-3810-Design-Practices-in-CS/project-1-BigOof`

Problem Statement The Interactive Delhi Metro Route Planner and Visualiser models the Delhi Metro network as a weighted, labelled graph in order to compute routes between two stations. The system supports two optimisation criteria: minimum travel time and minimum number of interchanges. The resulting route is presented through its station sequence, metro lines, interchange information, estimated travel time, and an interactive network visualisation.

Motivation The project applies graph-based route-finding algorithms to a practical transportation network while providing an interactive visualisation of the computed route. It provides a concrete application of shortest-path algorithms and graph data structures while separating the route computation from its presentation.

Intended Users The intended users are Delhi Metro passengers planning journeys between two stations, including regular commuters and visitors. The interface is designed so that users can obtain route information without requiring knowledge of the underlying graph representation or route-finding algorithms.

Main Goal The main goal of the application is to allow a user to select a source and destination station and obtain an optimal route according to the selected criterion: minimum travel time or minimum number of interchanges. The system then displays the corresponding station sequence, metro lines, interchange locations and count, estimated travel time, and highlighted route on the interactive map.

2 Correctness and Testing Plan

2.1 Unit Testing

ID	Unit tested	Test input / setup	Expected result	What it verifies
U1	`add_station()`	Add station A with name "Alpha" and line {`"Blue"`}	A appears in `stations` and `adjacency`; stored attributes are correct	Correct station creation
U2	`add_station()`	Add station A, then attempt to add A again	Raises `ValueError`	Duplicate station detection
U3	`add_connection()`	Add stations A and B, then connect A–B with time 4 on "Blue"	A contains B and vice versa, both with time 4 and line "Blue"	Correct bidirectional graph construction
U4	`add_connection()`	Attempt to connect station A to unknown station B	Raises `KeyError`	Invalid graph references are rejected
U5	`add_connection()`	Add connection A–B with travel time -1	Raises `ValueError`	Non-negative edge-weight requirement
U6	`fastestRoute()`	Two routes from A to D have total times 5 and 8	Returns the route with total time 5	Dijkstra selects minimum travel time
U7	`fastestRoute()`	Routes A–B–D and A–C–D; the route through B is found first, then the route through C gives a shorter total time	Returns the improved A–C–D route	Correct edge relaxation and predecessor update
U8	`fastestRoute()`	Source and destination are both station A	Returns `stations=(A,)`, `segments=()`, total time 0, interchanges 0	Correct zero-segment route
U9	`fastestRoute()`	Stations A and D have no connecting path	Raises `ValueError`	Correct handling of unreachable destinations
U10	`fewestInterchangesRoute()`	Two routes from A to D: one is faster but requires more line changes, while the other is slower with fewer changes	Returns the route with fewer interchanges	Primary interchange objective
U11	`fewestInterchangesRoute()`	Two routes from A to D have the same number of interchanges but different travel times	Returns the faster route	Lexicographic (interchanges, time) tie-breaking
U12	`fewestInterchangesRoute()`	Route A–B–C uses the same line on both consecutive edges	Same-line continuation does not increase interchange count	Correct interchange transition condition
U13	`fewestInterchangesRoute()`	Source and destination are both station A	Returns `stations=(A,)`, `segments=()`, total time 0, interchanges 0	Zero-segment behaviour for augmented routing
U14	`_result()` / reconstruction	Route A–B–C–D: A–B and B–C use Blue; C–D uses Red	Stations are `(A,B,C,D)`; total time is the sum of segment times; interchanges = 1	Correct route construction and interchange counting
U15	`fastestRoute()` / reconstruction	Route from A to D contains several segments	Returned station and segment sequences correspond exactly to the route	Correct predecessor reconstruction

Table 1: Unit tests for graph construction and route-finding functions.

2.2 Integration Tests

ID	Components integrated	Test scenario	Expected result
I1	Frontend + Flask API + Routing Engine	User selects a valid source and destination and requests **minimum travel time**	Frontend sends the correct request, Flask invokes `fastestRoute()`, and the optimal route is displayed.
I2	Frontend + Flask API + Routing Engine	User selects a valid source and destination and requests **minimum interchanges**	Flask invokes `fewestInterchangesRoute()`, and the frontend displays the route with the fewest interchanges.
I3	Routing Engine + Route Reconstruction + API	Routing algorithm finds a multi-segment route containing a line change	Reconstructed station and segment sequences, total travel time, and interchange count are correctly included in the API response.
I4	Data Layer + Routing Engine	Valid metro data is loaded and converted into the adjacency-list graph	All valid stations and connections are available to the routing algorithms, and a route can be computed using the loaded graph.
I5	API Validation + Routing Engine	Request contains an invalid station	Flask rejects the request without executing the routing algorithm.
I6	API Validation + Routing Engine	Request contains an unsupported routing mode	Flask rejects the request without invoking either routing algorithm.
I7	Routing Engine + Frontend	A second route request is made after a first route has been displayed	The new route replaces the previous route and its associated travel-time and interchange information.
I8	Routing Engine + Frontend Visualisation	A route contains one or more interchanges	The backend route is displayed with the correct station sequence and line changes on the metro map.

Table 2: Integration tests for the frontend, API, routing engine, and data layer.

2.3 Boundary Cases

Test inputs at the limits of the routing problem:

- **Source = destination:** zero-segment route, total time `0`, `0` interchanges.
- **Single connection:** route consists of exactly one segment.
- **No interchange:** entire route uses the same metro line.
- **Exactly one interchange:** route changes lines once.
- **Multiple interchanges:** route contains several line changes.
- **Direct vs. indirect route:** verify the algorithm handles the minimum possible number of segments correctly.
- **Unreachable destination:** no valid route exists.
- **Smallest valid graph:** two stations connected by one edge.
- **Multiple equal-cost routes:** verify the stated tie-breaking rule is respected.

2.4 Invalid-Input Testing

Verify that invalid requests are rejected before routing is performed:

- **Non-existent source station:**source code not present in the graph → reject with an appropriate error.
- **Non-existent destination station:**destination code not present → reject with an appropriate error.
- **Both stations invalid:**reject without invoking the routing algorithm.
- **Missing source/destination:**incomplete API request → validation error.
- **Missing routing mode:**request without `mode` → validation error.
- **Unsupported routing mode:**invalid mode value → reject rather than selecting an algorithm.
- **Invalid connection:**connection references an unknown station → `KeyError`.
- **Negative travel time:**connection with `travel_time < 0` → `ValueError`.
- **Missing/empty line:**connection without a valid line → `ValueError`.
- **Duplicate station code:**attempting to add an existing station → `ValueError`

2.5 Algorithm-Specific Test Cases

Fastest-route (Dijkstra):

- **Indirect route is faster:**direct connection exists, but a multi-edge route has lower total travel time → choose indirect route.
- **Relaxation required:**a station is initially reached through a more expensive path and later improved through a cheaper path → final route uses the improved distance.
- **Stale priority-queue entry:**an outdated queue entry remains after a distance is improved → stale entry is skipped and does not affect the result.
- **Equal travel-time routes:**two routes have the same total time → verify consistent handling of equal-cost paths.

Fewest-interchanges (state-augmented Dijkstra):

- **Fewer interchanges beats shorter time:**a route with fewer interchanges is selected even when it has greater travel time.
- **Travel time breaks interchange ties:**two routes have the same number of interchanges → route with lower travel time is selected.
- **Same-line continuation:**moving between stations on the same line adds `0` interchanges.
- **First segment:**boarding the first line from the source adds `0` interchanges.
- **Multiple arrival-line states:**the same station reached via different lines is treated as different states and can lead to different optimal routes.
- **Multiple interchanges:**verify the interchange count is accumulated correctly across several line changes.

2.6 UI Testing

ID	UI scenario	Expected result
UI-M1	Display a valid route	All stations and connections in the returned route are shown in the correct sequence.
UI-M2	Route contains multiple metro lines	Each route segment is displayed with its corresponding metro line.
UI-M3	Route contains an interchange	The interchange station is visually identifiable and the line change is represented correctly.

ID	UI scenario	Expected result
UI-M4	Display a route with multiple interchanges	All interchange points are displayed correctly.
UI-M5	Change the source or destination and submit a new route	The map updates to show only the newly calculated route.
UI-M6	Route contains many stations	The entire route remains visible and readable without incorrect station ordering or missing segments.
UI-M7	Source and destination are the same	No spurious route is displayed; the zero-segment result is handled correctly.
UI-M8	API returns an error or no route is available	No incorrect route is drawn and an appropriate error state is shown.
UI-M9	Route information is displayed alongside the map	Station sequence, travel time, and interchange count correspond to the route shown on the map.
UI-M10	Submit a route without selecting a valid source or destination	**w**The request is not submitted and the user is shown an appropriate input-validation message.

Table 3: UI tests for route visualisation and user interaction.

2.7 Performance Testing

ID	Scenario	Expected result
P1	Short route between nearby stations	Route is computed and returned promptly.
P2	Long route spanning many stations	Routing completes without significant increase in response time.
P3	Route with multiple interchanges	Augmented Dijkstra completes within an acceptable response time.
P4	Repeated route requests	Response time remains stable across consecutive requests.
P5	Multiple route requests on the full metro graph	System remains responsive and does not crash or time out.
P6	Map rendering of a long route	Complete route is rendered without noticeable lag or missing elements.

Table 4: Performance tests for route computation and visualisation.

3 Complexity and Performance Considerations

3.1 Time Complexity

The application performs station search, route computation, route reconstruction, and map visualisation. Searching through all stations takes $O(|V|)$ time using a linear scan, since each station may need to be checked against the query. An appropriate sorted index can reduce exact-search time to $O(\log |V|)$.

For minimum travel time, standard Dijkstra's algorithm with an adjacency list and binary-heap priority queue takes

$$O((|V| + |E|) \log |V|).$$

This is suitable because it explores connections efficiently while maintaining the shortest known travel time to each station.

For minimum interchanges, state-augmented Dijkstra takes

$$O(\bar{L}|E| \log(\bar{L}|V|)),$$

where $\bar{L}$ is the average number of metro lines per station. This additional factor arises because the algorithm tracks both the current station and the line used to arrive there. This is necessary to determine whether moving to the next station incurs an interchange. Route reconstruction and interchange detection take $O(k)$ time, where k is the number of stations in the computed route. Map-rendering time depends on the visualisation library and the number of elements displayed.

3.2 Space Complexity

The adjacency-list representation requires $O(|V| + |E|)$ space because it stores each station and its existing connections without allocating space for every possible station pair. Standard Dijkstra requires $O(|V|)$ auxiliary space for distances, predecessors, and the priority queue. State-augmented Dijkstra requires $O(\bar{L}|V|)$ auxiliary space because multiple arrival-line states may exist for each station. Reconstructing the route requires $O(k)$ additional space.

The map also consumes memory for station markers, connection segments, labels, and route highlighting. Keeping the graph in memory avoids repeatedly rebuilding it, trading a small persistent memory requirement for faster subsequent route requests.

3.3 Expected Input Size

The application operates on the finite Delhi Metro network, containing hundreds of stations and connections. Typically, each request consists of a source station, a destination station, and an optimisation criterion. The graph is loaded once during initialisation and reused across requests, avoiding repeated dataset parsing and graph construction.

Although individual requests are relatively small, the application should also be tested under increasing concurrency, targeting up to 100 simultaneous requests as specified in the project requirements. This helps evaluate whether computation and memory usage remain acceptable as demand increases.

3.4 Possible Bottlenecks

Repeated graph construction wastes time because the same station and connection data must be parsed and stored for every request. Linear station searches also become more expensive as the dataset grows, since more records must be examined before finding a match.

State-augmented Dijkstra processes more states and transitions than standard Dijkstra because it distinguishes arrival lines. This increases computation and memory usage, although the number of lines serving a station is generally small. Under high concurrency, multiple route calculations may compete for CPU time and memory, increasing response latency.

Map rendering can also become a bottleneck because displaying many station markers, connection segments, and labels requires additional graphical processing. Repeatedly redrawing unchanged elements or rendering elements outside the visible viewport can further increase rendering time and reduce interface responsiveness.

3.5 Maintaining UI Responsiveness

Route computation should not block the main UI thread, since long-running synchronous operations can prevent the interface from responding to station selections, resets, zooming, and panning. Where supported, route calculations should run in a background worker or separate execution thread, allowing the interface to remain interactive.

The graph should remain in memory between requests, and only the route information and map elements that have changed should be updated. Zooming and panning should modify the map viewport without triggering route recomputation. Input validation and appropriate feedback for invalid selections or failed calculations should also prevent incomplete results from being displayed.

3.6 Possible Optimisations and Trade-offs

An indexed station lookup can reduce search time, while an adjacency list and binary-heap priority queue provide efficient graph traversal and shortest-path computation. These choices offer good performance

without substantially increasing implementation complexity.

Frequently requested routes may be cached to avoid repeated computation, trading additional memory and cache-invalidation complexity for lower response times. Selective map updates and viewport-based rendering can reduce graphical processing, but may require more complex rendering logic. Background computation improves responsiveness but introduces additional coordination between the UI and route-finding module.

Response time, throughput, memory usage, and error rate should be measured under increasing concurrency to identify the most significant bottlenecks and balance computational efficiency, implementation complexity, memory use, and user experience.

4 Preliminary User Interface Design

4.1 Major Screen and Layout

The application will use a single-screen interface for the Interactive Delhi Metro Route Planner and Visualiser. As shown in the proposed mock-up (Figure 1), the screen is divided into four main areas: station selection, route optimisation controls, interactive metro map, and route details.

The station selection area contains source and destination dropdowns, with a swap button between them. Below these controls, users can choose between the fastest route and the route with the fewest interchanges. The central area displays the Delhi Metro network and highlights the computed route. The bottom area presents the ordered station sequence, associated metro lines, and interchange information in a scrollable panel.

This layout keeps route selection, visualisation, and journey details accessible without requiring navigation between screens.

4.2 Important Controls

- **Source and destination dropdowns:** Allow users to search for and select stations by name or station code.
- **Swap button:** Exchanges the source and destination to support planning the return journey.
- **Fastest Route:** Runs standard Dijkstra's algorithm to minimise estimated total travel time.
- **Least Interchanges:** Runs state-augmented Dijkstra to minimise line changes, using travel time to break ties.
- **Interactive metro map:** Displays metro lines and highlights the source, destination, and computed route. Zooming and panning allow users to inspect the network.
- **Route details panel:** Displays the ordered station sequence, metro lines, interchange stations, interchange count, and estimated travel time. The panel can be scrolled for longer journeys.
- **Reset control:** Clears the current selections and route information, allowing users to begin another search.

4.3 User Workflow

The user selects a source and destination station and chooses an optimisation criterion. The system validates the selections and computes the route using the corresponding algorithm. The resulting route is highlighted on the map, while the details panel displays the station sequence, metro lines, interchanges, and estimated travel time. The user can then swap stations, select another criterion, or reset the interface to plan another journey.

4.4 Feedback for Successful Operations

After a successful calculation, the interface highlights the computed route and marks its source and destination on the map. The details panel displays the complete station sequence, metro lines, interchange locations and count, and estimated travel time. The selected optimisation criterion should remain visually identifiable so users can distinguish between the fastest and fewest-interchanges routes.

4.5 Feedback for Errors

The interface will provide clear, contextual feedback for invalid selections and unsuccessful operations:

- **Missing station:** Prompt the user to select both a source and destination before calculating a route.
- **Invalid station:** Display a message when the search produces no valid station match.
- **Identical stations:** Request a different destination if the source and destination are identical.
- **No valid route:** Inform the user that no route is available for the selected pair.
- **Calculation failure:** Display an error and allow the user to retry or reset the interface.

Errors should be displayed near the relevant controls or in a clearly visible status area. Invalid or incomplete results should not replace a previously valid route until a new route has been computed successfully.

4.6 Responsiveness Considerations

The interface should remain responsive during route calculation and map interaction. Where supported, route computation will run outside the main UI thread to prevent blocking user input. The metro graph will be loaded once and reused across requests. Only changed route information and map elements should be updated, while zooming and panning should affect the viewport without triggering route recomputation. The route details panel will support scrolling to accommodate journeys of different lengths. The layout should adapt to smaller screens by stacking controls and results vertically where necessary.

4.7 Accessibility and Usability

Controls will have clear labels, consistent placement, and sufficiently large interactive areas. Station search should support both station names and station codes, and keyboard navigation should be supported wherever possible. Route distinctions must not rely solely on colour: the source, destination, and selected route should also use distinct markers, line styles, or textual labels. Text and controls should maintain sufficient contrast against their backgrounds. The station sequence and journey details provide a textual alternative to the map, allowing users to understand the route without relying exclusively on visual information.

4.8 Proposed UI Mock-up

Figure 1 presents the preliminary single-screen layout. It illustrates the placement of station selectors, route optimisation buttons, the interactive metro map, and the scrollable route details panel.

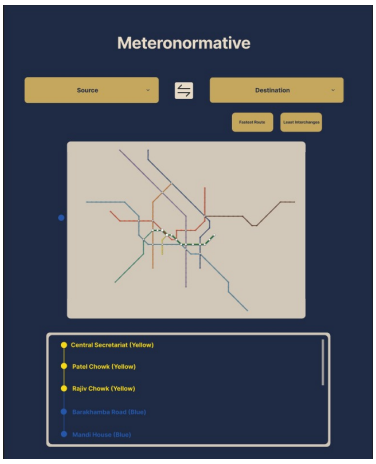


Figure 1: Preliminary single-screen interface for the Interactive Delhi Metro Route Planner and Visualiser.

5 Risks, Assumptions, and Limitations

5.1 Technical Risks and Mitigation

ID	Risk	Impact	Mitigation
R1	Incorrect or incomplete metro network data	Routes may be inaccurate, connections may be missing, and interchange counts may be incorrect.	Validate station identifiers, line assignments, and connections during initialisation. Detect duplicate or invalid connections and maintain version-controlled datasets.
R2	Incorrect route computation	The application may return a suboptimal route or incorrectly identify interchanges, reducing user trust.	Test both Dijkstra implementations against manually verified routes, including interchange-heavy routes, multiple-line stations, and unreachable destinations. Verify that travel-time weights are non-negative.
R3	Performance degradation under concurrent requests	Simultaneous route calculations may increase response times and memory usage, making the application less responsive.	Load the graph once, avoid unnecessary graph reconstruction, and test with increasing concurrency levels up to 100 requests. Measure response time, throughput, memory usage, and error rate.
R4	UI blocking or excessive map rendering	Route computation or rendering a large number of map elements may delay user interactions.	Separate route computation from UI logic, use background execution where supported, and update only the map elements that change.
R5	Invalid station selections or unexpected failures	Malformed inputs or calculation errors may interrupt the route-planning process or produce incomplete results.	Validate selections against the station dataset, reject identical source and destination stations, handle exceptions, and display clear error messages without presenting incomplete routes.

Table 5: Technical risks, impacts, and mitigation strategies

5.2 Assumptions

1. The station and connection dataset is sufficiently accurate and complete to represent the Delhi Metro network being modelled.
2. Estimated travel times between connected stations are non-negative and sufficiently representative for comparing routes. Real-time delays and service disruptions are not assumed to be available.
3. The network topology and line assignments remain relatively stable during a session, allowing the graph to be loaded once and reused across route requests.
4. Each station has a unique identifier, and connections reference valid stations and metro lines.
5. The expected workload consists primarily of individual source–destination queries, with concurrency testing performed up to 100 simultaneous requests.

5.3 Anticipated Limitations

1. **Dependence on dataset accuracy:** Route correctness depends on the completeness and accuracy of the stored stations, connections, line assignments, and travel-time estimates.

2. **Absence of real-time information:** Unless such data is integrated, the application cannot account for service disruptions, delays, temporary closures, or changing travel conditions.
3. **Simplified travel-time model:** Estimated travel time may exclude waiting times, platform transfers, walking distances within stations, and other sources of delay.
4. **Computational overhead:** State-augmented Dijkstra requires additional states and memory to track arrival lines, although the overhead is expected to remain manageable for the metro network.
5. **Scalability and visualisation constraints:** Performance under heavy concurrency and map-rendering load depends on the implementation, hardware, and visualisation library; the target of 100 concurrent requests must be validated experimentally.

6 Development Plan

The project will be developed incrementally, with each phase building on the functionality established in the previous phase.

Phase	Work	Responsible Member(s)	Expected Outcome
1	Review requirements and technical design	Gayatri and Shivanshi	Finalised requirements, module boundaries, graph representation, and implementation plan.
2	Core data structures / system layer	Shivanshi	Validated metro dataset, adjacency-list graph, and station lookup functionality.
3	Algorithms and core functionality	Gayatri	Working fastest-route and fewest-interchanges algorithms, including route reconstruction and interchange detection.
4	UI/UX integration	Gayatri and Shivanshi	Integrated interface for station selection, optimisation criteria, route details, and interactive map visualisation.
5	Testing and debugging	Gayatri and Shivanshi	Verified route correctness, error handling, responsiveness, and performance under increasing concurrency.
6	Final refinement and documentation	Gayatri and Shivanshi	Refined application, documented implementation and usage, and final testing results.

Table 6: Project development plan

6.1 Version Control and Collaboration

We use Git and GitHub for version control. During the building phase, we keep the process light so that we can make fast progress on separate components while keeping the repository working.

- **Repository and ownership.** We maintain one shared repository, and `main` must always run. Each of us owns different modules at any given time, following the development plan. We talk before editing the other person's files or changing a shared interface, such as the route result structure or the API contract.
- **Branching and pushing.** Small changes within our own modules are pushed directly to `main` after the code runs and the tests pass locally. Larger or riskier work, such as UI and API integration, is

done on a short-lived branch and merged once it works. We do not use GitHub issues or formal pull request reviews at this stage.

- **Commits.** We commit and push often, in small steps, with imperative messages that describe the change. We run the relevant tests before pushing.
- **Staying in sync.** We pull the latest changes before starting work and before each push. Whoever pushes resolves any conflict locally and re-runs the tests. If a conflict involves the other person's logic, we resolve it together rather than discarding either side's changes.
- **Checkpoints.** A GitHub Actions workflow runs the automated tests on every push to `main`, and failures are fixed or reverted promptly. At the end of each development phase, and before combining components, we review each other's work, check that it fits the agreed interfaces, and tag the milestone.
- **Testing and documentation phase.** Once the application is functionally complete, we tighten the process: bugs are tracked as GitHub issues, and changes to `main` go through pull requests with review.
- **Repository hygiene.** A `.gitignore` excludes environments, caches, and build artefacts. The dataset and a `README` with dependencies and run instructions are kept in the repository so the application can be reproduced from a fresh clone. Secrets are never committed.

7 Expected Deliverables

The final submission of the Interactive Delhi Metro Route Planner and Visualiser will consist of the following deliverables.

1. **Source code.** A version-controlled GitHub repository, organised into separate modules for network data, graph construction, route finding, route processing, input validation, the Flask API, and the map-based frontend, with a `README` documenting dependencies and build/run instructions.
2. **System and design documentation.** Documentation of the requirements, modular architecture, interfaces between modules, and the final user interface design.
3. **Description of data representations.** Documentation of the weighted, labelled graph $G = (V, E)$, the adjacency-list representation, the augmented state (u, ℓ), and the structure of the route result.
4. **Algorithm documentation.** Descriptions of standard Dijkstra for minimum travel time and state-augmented Dijkstra for minimum interchanges, including route reconstruction, interchange detection, and tie-breaking.
5. **Complexity analysis.** Time and space analysis of both routing algorithms and route reconstruction, together with measured performance under increasing concurrency.
6. **Demonstration of the major required features.** Screenshots or a short recording showing station search and selection, both routing criteria, route highlighting on the map, the route details panel, and error feedback.
7. **Representative sample inputs.** Source–destination pairs with the chosen criterion and expected outputs, covering routes with no, one, and multiple interchanges, as well as invalid and unreachable cases.
8. **Associated design assignments.** All design assignments completed for the project, consistent with the final implementation.

8 Design Review Questions

1. **What is the single most important algorithmic challenge in your project?**

   Computing the route with the fewest interchanges. Whether a move is an interchange depends on the line used to *arrive* at the current station, so a single scalar edge weight is insufficient. The algorithm must track the pair (u, ℓ) of station and arrival line, and minimise the pair (interchanges, travel time) lexicographically, while correctly handling stations served by several lines.

2. **What is the most appropriate data structure for the central problem, and why?**

   An adjacency list mapping each station code to its list of outgoing edges (v, w, c), together with a binary min-heap priority queue for Dijkstra. Each station connects to only a few neighbours, so the adjacency list uses $O(|V| + |E|)$ space rather than $O(|V|^2)$ and gives direct access to neighbours. The heap gives $O(\log |V|)$ extraction of the closest unprocessed state. For the augmented search, distance and predecessor maps are keyed by (u, ℓ).

3. **What is the most computationally expensive operation?**

   State-augmented Dijkstra, with time complexity $O(\bar{L}|E| \log(\bar{L}|V|))$ and $O(\bar{L}|V|)$ auxiliary space. It explores up to $\bar{L}$ times more states and transitions than standard Dijkstra, and the heap operations dominate. In practice, $\bar{L}$ is small and the Delhi Metro graph has only hundreds of stations, so this remains fast. Under high concurrency or for long routes, map rendering may become the more noticeable cost.

4. **What happens when the user provides invalid input?**

   Requests are validated against the station dataset *before* any routing algorithm is invoked. A missing, unknown, or malformed station code, identical source and destination, a missing or unsupported routing mode, and an incomplete request are all rejected with a clear error message. The frontend shows the message near the relevant controls and does not submit invalid selections. If no route exists, the user is told so. Previously displayed valid routes are not replaced by incomplete results. At the data layer, invalid graph construction raises `ValueError` (duplicate stations, negative times, empty lines) or `KeyError` (unknown stations).

5. **Which component is most likely to change during future development?**

   The interactive map and visualisation module. The route-finding algorithms are well defined and tested against manually verified routes, whereas the map depends on a visualisation library and on presentation choices that are more likely to be revised. Examples include switching the rendering library, moving from a schematic network diagram to a geographic map, changing how lines, interchanges, and the highlighted route are styled, or optimising rendering through viewport-based drawing or selective updates for performance.

6. **How does your architecture isolate that component?**

   The map module depends only on the structured route result returned by the Flask API (ordered station sequence, segments with their metro lines, interchange stations, total time, and interchange count) and on a separate station layout dataset used for drawing. It does not call, or depend on, any route-finding code, and the routing modules have no knowledge of the UI. The map is accessed through a small interface, such as drawing the network, highlighting a route, clearing the route, and fitting the view to a route. Replacing the library or changing the visual design therefore affects only this module. The graph, validation, routing algorithms, and API contract remain unchanged. Zooming and panning modify only the map viewport and never trigger route recomputation.

7. **What is one design decision you considered but rejected? Why?**

   Encoding interchanges as a single scalar weight, such as $w' = \text{time} + P \cdot \delta$ for a large penalty P, and running standard Dijkstra on that. We rejected this because the choice of P is arbitrary: too small, and a route with more interchanges can win, too large, and it behaves like the lexicographic rule only by coincidence. It also still requires tracking the arrival line to know when $\delta = 1$. State-augmented Dijkstra with exact lexicographic comparison is cleaner, provably minimises interchanges first, and uses travel time only as a tie-breaker.

8. **What additional feature could be implemented without redesigning the core architecture?**

   Several features fit the existing design: fare estimation (computed from the reconstructed route in the route-processing module), avoiding closed stations or lines (filtering edges before routing), alternative routes, and caching of frequently requested routes. Real-time delay data could also be added by updating edge weights in the graph layer, without changing the route-finding interface or the UI contract.